INTERNAL ASSIGNMENT

NAME	**SOHAIL ANSARI**
ROLL NO	**261450205242**
SESSION	**JULY 2026**
PROGRAM	**BACHELOR OF COMPUTER APPLICATIONS (BCA)**
SEMESTER	**I**
COURSE CODE & NAME	**DCA1109 & Introduction to Web Programming**

SET – 1

QUESTION -1: Explain HTML forms and their major elements. Discuss different input types and form attributes with suitable examples.

ANSWER - HTML Forms

An HTML form is a container used to collect input from users and send it to a server for processing. It acts as the interface between the user and the web application — for login pages, registration, search boxes, feedback, and online payments. A form is defined using the <form> element, and the data entered is transmitted to a server-side script specified in the action attribute, using the method named in the method attribute.

```
<form action="/register" method="post">
  <label for="uname">Name:</label>
  <input type="text" id="uname" name="username" required>
  <input type="submit" value="Register">
</form>
```

Major Form Elements

1. <form> — the container; holds all controls and defines where and how data is sent.
2. <input> — the most versatile control; its behaviour depends on the type attribute.
3. <label> — associates descriptive text with a control; the for attribute links to the control's id, improving accessibility and clickability.
4. <textarea> — a multi-line text box for comments or addresses.
5. <select> and <option> — create drop-down lists; <optgroup> groups related options.
6. <button> — a clickable button that can submit, reset, or trigger scripts.
7. <fieldset> and <legend> — group related controls and give the group a caption.
8. <datalist> — provides suggested values for a text input.
9. <output> — displays the result of a calculation.

Type	Purpose	Example
text	Single-line text	<input type="text" name="city">
password	Masked characters	<input type="password" name="pwd">
email	Validates e-mail format	<input type="email" name="mail">

Type	Purpose	Example
number	Numeric input with range	<input type="number" min="1" max="100">
radio	One option from many	<input type="radio" name="g" value="M">
checkbox	Zero or more options	<input type="checkbox" name="hobby">
date	Date picker	<input type="date" name="dob">
file	File upload	<input type="file" name="resume">
range	Slider	<input type="range" min="0" max="10">
hidden	Data not shown to user	<input type="hidden" name="id" value="7">
submit / reset	Submit or clear the form	<input type="submit" value="Send">

Common Input Types

Form Attributes

- action — URL of the server script that processes the data.
- method — GET (data appended to the URL, visible, limited length) or POST (data sent in the request body, suitable for sensitive or large data).
- name — identifies the form or control; essential for server-side retrieval of values.
- target — where to display the response (_blank, _self).
- enctype — encoding type; multipart/form-data is required for file uploads.
- novalidate — disables built-in browser validation.
- autocomplete — enables or disables browser suggestions.
- Input-level attributes: required, placeholder, readonly, disabled, maxlength, min/max, pattern (regular-expression validation), and autofocus.

Example Using Several Elements Together

```
<form action="/submit" method="post" enctype="multipart/form-data">
  <fieldset>
    <legend>Student Registration</legend>

    <label for="nm">Full Name:</label>
    <input type="text" id="nm" name="name" placeholder="Enter name" required>

    <label for="em">Email:</label>
    <input type="email" id="em" name="email" required>

    <label>Gender:</label>
    <input type="radio" name="gender" value="M"> Male
    <input type="radio" name="gender" value="F"> Female

    <label for="crs">Course:</label>
    <select id="crs" name="course">
      <option value="bca">BCA</option>
      <option value="bsc">B.Sc</option>
```

```
    </select>

    <label for="msg">Remarks:</label>
    <textarea id="msg" name="remarks" rows="3"></textarea>

    <button type="submit">Submit</button>
  </fieldset>
</form>
```

Forms matter because they make a web page interactive and data-driven; client-side validation attributes improve user experience and reduce unnecessary server load.

QUESTION -2: a) Explain JavaScript variables, data types and operators with suitable examples.
b) Explain how CSS is used to improve the appearance of a web page. Illustrate the use of simple CSS with suitable examples.

ANSWER - (a) Variables, Data Types and Operators in JavaScript

Variables are named containers for storing data. In modern JavaScript, three keywords are used: var (function-scoped, hoisted), let (block-scoped, reassignable), and const (block-scoped, cannot be reassigned). let and const are preferred as they avoid scoping bugs associated with var.

```
let name = "Rahul";
const pi = 3.14;
var age = 20;
age = 21;          // allowed
// pi = 3.15;      // error: const cannot be reassigned
```

Data Types fall into two groups. Primitive types are: String (text), Number (integers and floats), Boolean (true/false), Undefined (declared but unassigned), Null (intentional absence of value), BigInt (large integers), and Symbol (unique identifiers). Non-primitive type is Object, which includes arrays, functions, and date objects. JavaScript is dynamically typed, so a variable's type is determined at runtime.

```
let s = "Hello";        // String
let n = 42;             // Number
let b = true;           // Boolean
let u;                  // Undefined
let x = null;           // Null
let arr = [1, 2, 3];    // Object (Array)
let obj = {a: 1};       // Object
console.log(typeof s);  // "string"
```

Operators are symbols that perform operations on operands:

- Arithmetic: + - * / % ** — e.g., 10 % 3 gives 1.
- Assignment: = += -= *= /= — e.g., x += 5 adds 5 to x.
- Comparison: == != === !== > < >= <=. The strict equality === compares both value and type, so "5" == 5 is true but "5" === 5 is false.
- Logical: && (AND), || (OR), ! (NOT).
- Increment/Decrement: ++, --.
- Ternary: condition ? value1 : value2.
- Type: typeof, instanceof.

```
let a = 10, b = 3;
console.log(a + b);             // 13
```

```
console.log(a === "10");         // false
console.log(a > b && b > 0);     // true
console.log(a % 2 === 0 ? "Even" : "Odd");  // "Even"
```

Understanding variables, types, and operators is fundamental, since every JavaScript program — from simple validation to complex applications — is built on them.

(b) Role of CSS in Improving Web Page Appearance

CSS (Cascading Style Sheets) is a style-sheet language used to control the presentation of HTML documents. It separates content (HTML) from presentation (CSS), which makes pages easier to maintain, reduces code duplication, and enables consistent design across an entire website. A single external style sheet can restyle thousands of pages by changing just one file.

CSS improves appearance in several concrete ways:

1. Typography control — font family, size, weight, line-height, letter-spacing, and text alignment make content readable and attractive.
2. Colour and background — text colour, background colours, and gradients create visual hierarchy and brand identity.
3. Layout and positioning — properties such as display, flex, grid, margin, and padding arrange elements into clean, responsive structures.
4. Borders, shadows, and rounded corners — add depth and modern visual appeal.
5. Responsiveness — media queries adapt the layout to different screen sizes, improving usability on mobile devices.
6. Interactivity feedback — pseudo-classes such as :hover and :focus provide visual cues.
7. Consistency — class-based styling ensures uniform appearance across pages.

CSS can be applied in three ways: inline (via the style attribute), internal (inside a <style> block in the <head>), and external (a separate .css file linked using <link>). External CSS is generally preferred for larger sites.

Simple CSS Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      margin: 20px;
    }
    h1 {
      color: #ffffff;
      background-color: #00509e;
      padding: 10px;
      border-radius: 6px;
      text-align: center;
    }
    p {
      color: #333333;
      line-height: 1.6;
      text-align: justify;
    }
    .btn {
```

```
    background-color: #00509e;
    color: white;
    padding: 8px 16px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
   }
   .btn:hover { background-color: #003366; }
  </style>
</head>
<body>
  <h1>Welcome to My Page</h1>
  <p>This paragraph is styled using CSS to improve readability.</p>
  <button class="btn">Click Me</button>
</body>
</html>
```

Here the heading gets a coloured banner, the paragraph gets comfortable line spacing and justified alignment, and the button changes colour on hover — all without altering the HTML content. Thus CSS transforms plain semantic markup into a visually organised, accessible, and responsive user interface.

QUESTION -3: Explain the basic structure of an HTML document. Also discuss the features and uses of HTML with suitable examples.

ANSWER - Basic Structure of an HTML Document

Every HTML5 document follows a standard skeleton with a document type declaration followed by the html element, which contains two main sections — the head and the body.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Sample page">
  <title>My First Web Page</title>
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
<body>
  <header><h1>Welcome</h1></header>
  <nav><a href="#about">About</a></nav>
  <main>
    <section id="about">
      <h2>About</h2>
      <p>This is a paragraph.</p>
    </section>
  </main>
  <footer><p>&copy; 2026</p></footer>
</body>
</html>
```

Explanation of each part:

1. <!DOCTYPE html> — declares the document as HTML5; ensures standards mode rendering by the browser.
2. <html> — the root element; lang="en" specifies the document language, aiding screen readers and search engines.
3. <head> — contains metadata not displayed on the page: <meta charset> for character encoding, <meta name="viewport"> for responsive rendering on mobile devices, <meta name="description"> for search-engine snippets, <title> (shown on the browser tab), linked CSS files, and script files.
4. <title> — mandatory; appears in the tab, bookmarks, and search results.
5. <body> — holds all visible content: headings, paragraphs, images, links, tables, forms, and semantic sections such as <header>, <nav>, <main>, <section>, <article>, and <footer>.

The separation of head (metadata) and body (visible content) is the defining feature of the structure; the browser parses the document top-down and builds the DOM (Document Object Model), which scripts can then manipulate.

Features of HTML

1. Easy to learn and use — simple tag-based syntax; readable by beginners.
2. Platform independence — the same file renders on any operating system and any browser.
3. Markup, not programming — HTML describes structure and semantics rather than logic.
4. Hypertext capability — the <a> element links documents across the web, which is the foundation of the World Wide Web.
5. Semantic elements — tags such as <header>, <article>, and <section> convey meaning to browsers, assistive technologies, and search engines.

Uses of HTML

- Building web pages and websites — from static home pages to corporate portals.
- Creating web applications — combined with CSS and JavaScript for dynamic interfaces.
- Email templates — HTML is used for formatted newsletters.
- Documentation and e-books — structured documents viewable in any browser.
- Offline documentation — bundled HTML help files.
- Embedding media — inserting video, audio, maps via iframes.
- Data collection — forms for registration, surveys, and payments.
- Prototyping — rapid mock-ups of interfaces.

Simple example demonstrating structure and features:

```
<!DOCTYPE html>
<html>
<head><title>Product Page</title></head>
<body>
  <h1>Wireless Headphones</h1>
  <img src="headphones.jpg" alt="Wireless headphones" width="200">
  <p>Price: <strong>₹2,499</strong></p>
  <a href="https://example.com/buy">Buy Now</a>
  <video src="demo.mp4" controls width="300"></video>
</body>
</html>
```

This example shows semantic headings, an image with an accessibility-friendly alt attribute, a hyperlink, and native multimedia — illustrating how HTML provides structure, meaning, and media embedding. HTML is therefore the backbone of the web: it defines content and structure,

while CSS handles presentation and JavaScript handles behaviour, together forming the three core technologies of web development.

SET – 2

QUESTION -4: (A) EXPLAIN JAVASCRIPT FUNCTIONS, OBJECTS AND ARRAYS WITH SUITABLE EXAMPLES. ALSO MENTION THE ROLE OF JAVASCRIPT DEBUGGERS.
(b) What is JSON? Explain JSON data types, parsing and stringify operations with suitable examples.

ANSWER - (a) Functions, Objects and Arrays in JavaScript

Functions are reusable blocks of code that perform a specific task and can accept parameters and return values. They can be declared, expressed, or defined as arrow functions.

javascript

```
// Function declaration
function add(a, b) { return a + b; }

// Function expression
const sub = function(a, b) { return a - b; };

// Arrow function
const mul = (a, b) => a * b;

console.log(add(5, 3));   // 8
console.log(mul(4, 2));   // 8
```

Functions support default parameters (function greet(name = "Guest")), rest parameters (...nums), and callbacks, and they create their own scope. They are first-class objects, so they can be assigned to variables and passed as arguments.

Objects are collections of key–value pairs used to model real-world entities. Values can be primitives, arrays, or other objects, and properties can hold functions (methods).

```
let student = {
  name: "Amit",
  age: 20,
  course: "BCA",
  greet: function() { return "Hi, I am " + this.name; }
};
console.log(student.name);       // Amit
console.log(student["age"]);     // 20
console.log(student.greet());    // Hi, I am Amit
student.age = 21;                // modify
student.city = "Delhi";          // add new property
```

Objects are commonly created with object literals, constructors, Object.create(), or ES6 classes (class Student { constructor(name){ this.name = name; } }). Iteration over keys is done with for...in or Object.keys().

Arrays are ordered, index-based collections of values, created with literals [] or new Array(). They support a rich set of methods.

```
let marks = [78, 85, 62, 90];
console.log(marks[0]);           // 78
marks.push(95);                  // add at end
marks.pop();                     // remove from end
marks.unshift(50);               // add at start
```

```
console.log(marks.length);       // 4
console.log(marks.map(m => m + 5));       // transform each element
console.log(marks.filter(m => m >= 80));  // select elements
console.log(marks.reduce((a, b) => a + b, 0)); // sum
```

Arrays in JavaScript can hold mixed types and are themselves objects; methods such as forEach, find, slice, and sort make data processing concise.

Role of JavaScript Debuggers

Debuggers are tools — built into browsers (Chrome DevTools, Firefox Developer Tools) and IDEs (VS Code) — that help find and fix errors. Their key functions are: setting breakpoints to pause execution at a chosen line; step-over, step-into, and step-out to trace execution flow; inspecting variables and the call stack at runtime; watching expressions; using the console for logging (console.log, console.error, console.table); and analysing performance. Debugging is essential because JavaScript errors can be silent — the code runs but produces wrong output — and the debugger makes the internal state of the program visible, drastically reducing development time.

(b) JSON: Definition, Data Types, Parsing and Stringify

JSON (JavaScript Object Notation) is a lightweight, text-based data-interchange format derived from JavaScript object syntax. It is language-independent, human-readable, and widely used to exchange data between a client and a server, especially in REST APIs and AJAX calls. JSON files use the .json extension and the MIME type application/json.

JSON Data Types

JSON supports a limited set of data types:

1. String — text in double quotes, e.g., "Rahul" (single quotes are not allowed).
2. Number — integer or floating point, e.g., 42, -7.5, 1.2e3 (no quotes).
3. Boolean — true or false.
4. Null — null represents absence of value.

JSON does not support functions, undefined, comments, dates, or symbols — these must be represented as strings or omitted.

Example JSON document:

```
{
  "id": 101,
  "name": "Amit Sharma",
  "email": "amit@example.com",
  "active": true,
  "address": null,
  "courses": ["BCA", "Web Programming"],
  "marks": { "web": 88, "dbms": 79 }
}
```

Parsing (JSON string → JavaScript object)

JSON.parse() converts a JSON string into a usable JavaScript object, enabling access to its properties.

javascript

```
const jsonText = '{"name":"Amit","age":20,"courses":["BCA","WP"]}';
const obj = JSON.parse(jsonText);

console.log(obj.name);       // Amit
console.log(obj.courses[1]);  // WP
```

JSON.parse() also accepts a reviver function to transform values during parsing, e.g., converting a date string to a Date object:

javascript
```
const d = JSON.parse('{"dob":"2026-01-15"}',
  (key, value) => key === "dob" ? new Date(value) : value);
```
A syntax error throws an exception, so parsing untrusted data should be wrapped in try...catch.
Stringify (JavaScript object → JSON string)
JSON.stringify() converts a JavaScript value into a JSON string, which is required when sending data to a server, storing in localStorage, or writing to a file.
```
const student = {
  name: "Amit",
  age: 20,
  greet: function() { return "hi"; }   // functions are omitted
};

const str = JSON.stringify(student);
console.log(str);   // {"name":"Amit","age":20}
```
JSON.stringify() supports two optional arguments: a replacer (array of property names or a function to filter) and a space argument for indentation.
javascript
```
console.log(JSON.stringify(student, ["name"], 2));
// {
//   "name": "Amit"
// }

console.log(JSON.stringify({a:1, b:2}, (k, v) => v * 2));
// {"a":2,"b":4}
```
Note that JSON.stringify() ignores functions, undefined, and symbols, and converts dates to ISO strings. For deep cloning, JSON.parse(JSON.stringify(obj)) is a common technique, though it loses functions and dates.
Summary: JSON provides a compact, standard format for data exchange; JSON.parse() and JSON.stringify() are the two bridge functions that connect the text format with live JavaScript objects, making JSON indispensable in modern web development.

QUESTION -5: Discuss the important features of a modern e-commerce website. Explain e-business models, e-marketing, online payment security, use of REST APIs for fetching data and the importance of web accessibility guidelines.
ANSWER - Important Features of a Modern E-commerce Website
A modern e-commerce site must deliver a fast, secure, and personalised shopping experience. Key features include:

1. Product catalogue with rich media — high-quality images, zoom, video, specifications, and reviews.
2. Powerful search and filtering — faceted search by price, brand, size, and rating.
3. Shopping cart and checkout — persistent cart, guest checkout, multiple addresses, and order summary.
4. User accounts and wishlists — order history, saved addresses, and recommendations.
5. Multiple payment options — cards, UPI, net banking, wallets, and cash on delivery.
6. Order tracking and notifications — real-time shipping status and e-mail/SMS alerts.
7. Mobile responsiveness — mobile-first design, as a large share of traffic is mobile.
8. Personalisation and recommendations — “customers also bought”, recently viewed items.

E-business Models

- B2C (Business to Consumer): a business sells directly to end consumers, e.g., Amazon, Flipkart.
- B2B (Business to Business): transactions between companies, e.g., IndiaMART, Alibaba.
- C2C (Consumer to Consumer): consumers sell to each other through a platform, e.g., OLX, eBay.
- C2B (Consumer to Business): individuals offer services to businesses, e.g., freelancers on Upwork.
- B2G / G2C: business or government interaction with citizens, e.g., government portals.
- D2C (Direct to Consumer): brands sell directly, bypassing intermediaries.
- Subscription and marketplace models also co-exist within these categories.

E-marketing

E-marketing is the use of digital channels to promote products and build customer relationships. Techniques include search engine optimisation (SEO), search engine marketing (SEM/PPC), social media marketing, e-mail campaigns, content marketing and blogs, affiliate marketing, influencer marketing, retargeting ads, and analytics-driven personalisation. Its advantages are measurability, targeting, low cost per reach, and the ability to personalise at scale; challenges include ad fatigue, privacy regulation, and intense competition.

Online Payment Security

Security is critical because payment data is highly sensitive. Standard practices are:

- HTTPS/TLS encryption for all traffic.
- PCI-DSS compliance by payment gateways.
- Tokenisation — replacing card numbers with tokens.
- Two-factor authentication (2FA) and OTP verification.
- Secure payment gateways such as Razorpay, PayU, or Stripe, so the merchant never stores card data.

Use of REST APIs for Fetching Data

Modern e-commerce sites are often decoupled — the frontend (React, Angular, or a mobile app) fetches data from a backend through REST APIs. REST (Representational State Transfer) uses standard HTTP methods on resource URLs: GET /api/products to list products, GET /api/products/101 for one product, POST /api/cart to add to cart, PUT to update, and DELETE to remove. Responses are typically JSON. Benefits include separation of concerns, reusability across web and mobile clients, scalability, caching, and easier integration with third-party services such as payment gateways and logistics providers. APIs are usually secured using tokens (JWT/OAuth) and rate limiting.

Importance of Web Accessibility Guidelines

Accessibility ensures that people with disabilities — visual, auditory, motor, or cognitive — can use the site. The WCAG (Web Content Accessibility Guidelines) are organised around four principles: Perceivable, Operable, Understandable, and Robust (POUR). Practical measures include alternative text for images, keyboard navigability, sufficient colour contrast, semantic HTML and ARIA labels, captions for video, resizable text, and clear form labels.

Accessibility matters because it is inclusive (about 15–20% of people live with some disability), it is often a legal requirement, it expands the customer base, improves SEO (semantic markup is favoured by search engines), and generally produces cleaner, more maintainable code. An inaccessible e-commerce site excludes potential customers and risks reputational and legal consequences.

QUESTION-6: (A) EXPLAIN HTTP AND ITS COMMONLY USED METHODS. ALSO DISCUSS IMPORTANT HTTP STATUS CODES WITH SUITABLE EXAMPLES.

(b) What are cookies? Explain how cookies can be created and read using JavaScript.

ANSWER - (a) HTTP and Its Methods

HTTP (HyperText Transfer Protocol) is an application-layer protocol used for communication between a web client (browser) and a web server. It follows a request–response model: the client sends a request containing a method, a URL, headers, and optionally a body; the server responds with a status code, headers, and a body (usually HTML, JSON, or an image). HTTP is stateless, meaning each request is independent — state is maintained using cookies, sessions, or tokens. HTTPS is HTTP over TLS, which encrypts the communication.

Commonly Used HTTP Methods

Method	Purpose	Example
GET	Retrieve a resource; no body; safe and idempotent	GET /products
POST	Submit data to create a resource	POST /register with form da
PUT	Replace a resource entirely	PUT /users/5
PATCH	Partially update a resource	PATCH /users/5

GET appends data to the URL as a query string and is limited in length; POST sends data in the request body, is not visible in the URL, and is preferred for sensitive or large data. PUT and DELETE are idempotent — repeating them yields the same result, unlike POST.

Important HTTP Status Codes

Status codes are three-digit numbers grouped into five classes: 1xx informational, 2xx success, 3xx redirection, 4xx client error, 5xx server error.

- 200 OK — request succeeded; the response contains the resource.
- 201 Created — a new resource was created, typically after POST.
- 204 No Content — success but no body returned, common for DELETE.
- 301 Moved Permanently — the resource has a new permanent URL.
- 302 Found — temporary redirect.
- 304 Not Modified — the cached copy is still valid (used with conditional requests).
- 400 Bad Request — malformed syntax or invalid parameters.
- 401 Unauthorized — authentication required or credentials invalid.
- 403 Forbidden — authenticated but not permitted to access.

Example: A login attempt with wrong credentials typically returns 401, a request to a non-existent product returns 404, and a successful product creation returns 201 with the new resource's URL in the Location header. Status codes thus provide a standardised, machine-readable way for clients to interpret the outcome of requests.

(b) Cookies: Creation and Reading with JavaScript

Cookies are small pieces of text data (up to about 4 KB each) that a website stores in the user's browser and that are automatically sent back to the server with every subsequent request to the same domain. They are the classical mechanism for maintaining state in the otherwise stateless HTTP protocol — used for session management (login sessions, shopping carts), personalisation (language, theme), and tracking (analytics, advertising).

A cookie has several attributes: name, value, domain, path, expires or max-age, Secure (sent only over HTTPS), HttpOnly (inaccessible to JavaScript, preventing XSS theft), and SameSite (controls cross-site sending, mitigating CSRF).

Creating a Cookie

Cookies are created and read through the document.cookie property. Assigning a string of the form name=value; expires=...; path=... creates or updates a cookie.

javascript

```
// Simple cookie (session cookie, deleted when browser closes)
document.cookie = "username=Amit";

// Cookie with expiry date (persistent cookie)
const d = new Date();
d.setTime(d.getTime() + (7 * 24 * 60 * 60 * 1000)); // 7 days
document.cookie = "theme=dark; expires=" + d.toUTCString() + "; path=/";

// Secure and SameSite attributes
document.cookie = "token=abc123; path=/; Secure; SameSite=Strict";
```

Important points: the path attribute should usually be set to / so the cookie is available site-wide; without expires or max-age the cookie is a session cookie; special characters in values must be URI-encoded using encodeURIComponent().

Reading a Cookie

document.cookie returns all cookies for the current path as a single string, separated by "; ". There is no built-in method to fetch one cookie by name, so the string must be split and searched.

```
function getCookie(name) {
  const parts = document.cookie.split("; ");
  for (let part of parts) {
    const [key, value] = part.split("=");
    if (key === name) {
      return decodeURIComponent(value);
    }
  }
  return null;   // not found
}

console.log(getCookie("username"));  // "Amit"
console.log(getCookie("theme"));     // "dark"
```

A shorter alternative uses document.cookie.split("; ") with find():

```
const theme = document.cookie
  .split("; ")
  .find(row => row.startsWith("theme="))
  ?.split("=")[1];
```

Updating and Deleting Cookies

Updating is done by setting the same cookie name again with a new value. Deletion is achieved by setting the same name and path with an expiry date in the past:

```
document.cookie = "theme=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/";
```

Limitations and Best Practices

Cookies are limited to about 4 KB and roughly 50 per domain; they are sent with every request, adding overhead. Since JavaScript-accessible cookies can be stolen by cross-site scripting, sensitive session identifiers should be stored in cookies marked HttpOnly and Secure, and served over HTTPS. For larger or non-transmitted client-side storage, localStorage and sessionStorage are more appropriate, though they are also JavaScript-accessible. Cookies remain essential, but must be used with security attributes and privacy compliance in mind.